

CMSIS

```
typedef enum IRQn {
    Cortex-M3 Processor Exceptions Numbers
        NonMaskableInt_IRQn    = -14
        MemoryManagement_IRQn  = -12
        BusFault_IRQn           = -11
        SVCall_IRQn             = -5
        DebugMonitor_IRQn       = -4
        PendSV_IRQn             = -2
        SysTick_IRQn            = -1
    LPC17xx Specific Interrupt Numbers
        WDT_IRQn                = 0
        TIMER0_IRQn             = 1
        TIMER1_IRQn             = 2
        TIMER2_IRQn             = 3
        TIMER3_IRQn             = 4
        UART0_IRQn              = 5
        UART1_IRQn              = 6
        UART2_IRQn              = 7
        UART3_IRQn              = 8
        PWM1_IRQn               = 9
        I2C0_IRQn               = 10
        I2C1_IRQn               = 11
        I2C2_IRQn               = 12
        SPI_IRQn                = 13
        SSP0_IRQn               = 14
        SSP1_IRQn               = 15
        PLL0_IRQn               = 16
        RTC_IRQn                = 17
        EINT0_IRQn              = 18
        EINT1_IRQn              = 19
        EINT2_IRQn              = 20
        EINT3_IRQn              = 21
        ADC_IRQn                = 22
        BOD_IRQn                = 23
        USB_IRQn                = 24
        CAN_IRQn                = 25
        DMA_IRQn                = 26
        I2S_IRQn                = 27
        ENET_IRQn               = 28
        RIT_IRQn                = 29
        MCPWM_IRQn              = 30
        QEI_IRQn                = 31
        PLL1_IRQn               = 32
        USBActivity_IRQn        = 33
        CANActivity_IRQn        = 34
} IRQn_Type;

void SysTick_Handler(void);
void WDT_IRQHandler(void);
void TIMER0_IRQHandler(void);
void TIMER1_IRQHandler(void);
void TIMER2_IRQHandler(void);
void TIMER3_IRQHandler(void);
void UART0_IRQHandler(void);
void UART1_IRQHandler(void);
void UART2_IRQHandler(void);
void UART3_IRQHandler(void);
void PWM1_IRQHandler(void);
void I2C0_IRQHandler(void);
void I2C1_IRQHandler(void);
void I2C2_IRQHandler(void);
void SPI_IRQHandler(void);
void SSP0_IRQHandler(void);
void SSP1_IRQHandler(void);
void PLL0_IRQHandler(void);
void RTC_IRQHandler(void);
void EINT0_IRQHandler(void);
void EINT1_IRQHandler(void);
void EINT2_IRQHandler(void);
void EINT3_IRQHandler(void);
void ADC_IRQHandler(void);
void BOD_IRQHandler(void);
void USB_IRQHandler(void);
void CAN_IRQHandler(void);
void DMA_IRQHandler(void);
void I2S_IRQHandler(void);
void ENET_IRQHandler(void);
void RIT_IRQHandler(void);
void MCPWM_IRQHandler(void);
void QEI_IRQHandler(void);
void PLL1_IRQHandler(void);
void USBActivity_IRQHandler(void);
void CANActivity_IRQHandler(void);

typedef unsigned char    uint8_t;
typedef unsigned short   uint16_t;
typedef unsigned          uint32_t;

    @param IRQn    Number of the external interrupt to enable
void NVIC_EnableIRQ(IRQn_Type IRQn);

    @param IRQn    Number of the external interrupt to enable
void NVIC_DisableIRQ(IRQn_Type IRQn);

    @param IRQn    Number of the external interrupt to enable
void NVIC_ClearPendingIRQ(IRQn_Type IRQn);

    @param IRQn    Number of the external interrupt to enable
    @param priority Priority to set
void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority);

    @param ticks    Number of ticks between two interrupts
    @return    0    Function succeeded
    @return    1    Function failed
uint32_t SysTick_Config(uint32_t ticks);
```

```

typedef struct {
    __IO uint32_t FIODIR;
    __IO uint32_t FIOMASK;
    __IO uint32_t FIOPIN;
    __IO uint32_t FIOSET;
    __IO uint32_t FIOCLR;
} LPC_GPIO_TypeDef;
#define LPC_GPIO0
#define LPC_GPIO1
#define LPC_GPIO2
#define LPC_GPIO3
#define LPC_GPIO4

typedef struct {
    __IO uint32_t PCONP;
    __IO uint32_t EXTINT;
    __IO uint32_t EXTMODE;
    __IO uint32_t EXTPOLAR;
    __IO uint32_t PCLKSEL0;
    __IO uint32_t PCLKSEL1;
    __IO uint32_t DMAREQSEL;
} LPC_SC_TypeDef;
#define LPC_SC

typedef struct {
    __IO uint32_t PINSEL0;
    :
    __IO uint32_t PINSEL10;
    :
    __IO uint32_t PINMODE9;
} LPC_PINCON_TypeDef;
#define LPC_PINCON

typedef struct {
    __I uint32_t IntStatus;
    __I uint32_t IO0IntStatR;
    __I uint32_t IO0IntStatF;
    __O uint32_t IO0IntClr;
    __IO uint32_t IO0IntEnR;
    __IO uint32_t IO0IntEnF;
    __I uint32_t IO2IntStatR;
    __I uint32_t IO2IntStatF;
    __O uint32_t IO2IntClr;
    __IO uint32_t IO2IntEnR;
    __IO uint32_t IO2IntEnF;
} LPC_GPIOINT_TypeDef;
#define LPC_GPIOINT

typedef struct {
    __IO uint32_t ADCR;
    __IO uint32_t ADGDR;
    uint32_t RESERVED0;
    __IO uint32_t ADINTEN;
    __I uint32_t ADDR0;
    __I uint32_t ADDR1;
    __I uint32_t ADDR2;
    __I uint32_t ADDR3;
    __I uint32_t ADDR4;
    __I uint32_t ADDR5;
    __I uint32_t ADDR6;
    __I uint32_t ADDR7;
    __I uint32_t ADSTAT;
    __IO uint32_t ADTRM;
} LPC_ADC_TypeDef;
#define LPC_ADC

```

```

typedef struct {
    __IO uint32_t IR;
    __IO uint32_t TCR;
    __IO uint32_t TC;
    __IO uint32_t PR;
    __IO uint32_t PC;
    __IO uint32_t MCR;
    __IO uint32_t MR0;
    __IO uint32_t MR1;
    __IO uint32_t MR2;
    __IO uint32_t MR3;
    __IO uint32_t CCR;
    __I uint32_t CR0;
    __I uint32_t CR1;
    __IO uint32_t EMR;
    __IO uint32_t CTCR;
} LPC_TIM_TypeDef;
#define LPC_TIM0
#define LPC_TIM1
#define LPC_TIM2
#define LPC_TIM3

typedef struct {
    __IO uint32_t DACR;
    __IO uint32_t DACCTRL;
    __IO uint16_t DACCNTVAL;
} LPC_DAC_TypeDef;
#define LPC_DAC

typedef struct {
    __I uint32_t DMACIntStat;
    __I uint32_t DMACIntTCStat;
    __O uint32_t DMACIntTCClear;
    __I uint32_t DMACIntErrStat;
    __O uint32_t DMACIntErrClr;
    __I uint32_t DMACRawIntTCStat;
    __I uint32_t DMACRawIntErrStat;
    __I uint32_t DMACEnbldChns;
    __IO uint32_t DMACSoftBReq;
    __IO uint32_t DMACSoftSReq;
    __IO uint32_t DMACSoftLBReq;
    __IO uint32_t DMACSoftLSReq;
    __IO uint32_t DMACConfig;
    __IO uint32_t DMACSync;
} LPC_GPDMA_TypeDef;
#define LPC_GPDMA

typedef struct {
    __IO uint32_t DMACCSrcAddr;
    __IO uint32_t DMACCDestAddr;
    __IO uint32_t DMACCLLI;
    __IO uint32_t DMACControl;
    __IO uint32_t DMACConfig;
} LPC_GPDMA_TypeDef;
#define LPC_GPDMA0
#define LPC_GPDMA1
#define LPC_GPDMA2
#define LPC_GPDMA3
#define LPC_GPDMA4
#define LPC_GPDMA5
#define LPC_GPDMA6
#define LPC_GPDMA7

```

DRIVERS

COMMON TYPES

```
typedef enum {  
    FALSE = 0,  
    TRUE  = !FALSE  
} Bool;
```

```
typedef enum {  
    RESET = 0,  
    SET   = !RESET  
} FlagStatus, IntStatus, SetState;
```

```
typedef enum {  
    DISABLE = 0,  
    ENABLE  = !DISABLE  
} FunctionalState;
```

```
typedef enum {  
    ERROR   = 0,  
    SUCCESS = !ERROR  
} Status;
```

```
typedef enum {  
    NONE_BLOCKING = 0,  
    BLOCKING  
} TRANSFER_BLOCK_Type;
```

```
typedef enum {  
    PORT_0 = 0,  
    :  
    PORT_4  
} LPC_PORT;
```

```
typedef enum {  
    PIN_0 = 0,  
    :  
    PIN_31  
} LPC_PIN;
```

PINSEL

```
typedef enum {  
    PINSEL_FUNC_00 = 0,  
    PINSEL_FUNC_01 = 1,  
    PINSEL_FUNC_10 = 2,  
    PINSEL_FUNC_11 = 3  
} PINSEL_FUNC;
```

```
typedef enum {  
    PINSEL_PULLUP = 0,  
    PINSEL_REPEATER,  
    PINSEL_TRISTATE,  
    PINSEL_PULLDOWN  
} PINSEL_MODE;
```

```
typedef struct {  
    LPC_PORT      port;  
    LPC_PIN       pin;  
    PINSEL_FUNC   func;  
    PINSEL_MODE   mode;  
    FunctionalState openDrain;  
} PINSEL_CFG_T;
```

@brief Configures the function, resistor mode, and open-drain setting for a single pin.

@param pinCfg Pointer to a PINSEL_CFG_T structure containing the pin configuration.

```
void PINSEL_ConfigPin(const PINSEL_CFG_T* pinCfg);
```

@brief Configures multiple pins on the same port with identical settings.

@param pinCfg Pointer to a PINSEL_CFG_T configuration structure (pin field is ignored).

@param pinMask A 32-bit mask representing the pins to be configured.

```
void PINSEL_ConfigMultiplePins(const PINSEL_CFG_T* pinCfg, uint32_t pinMask);
```

GPIO

```
typedef enum {
    GPIO_INPUT = 0,
    GPIO_OUTPUT
} GPIO_DIR;

typedef enum {
    GPIO_INT_RISING = 0,
    GPIO_INT_FALLING
} GPIO_INT_EDGE;

@brief Sets the direction for a group of pins on a specific port.
@param port The GPIO port (PORT_x [0...4]).
@param pinMask A 32-bit mask representing the pins to modify.
@param dir The direction (GPIO_INPUT or GPIO_OUTPUT).
void GPIO_SetDir(LPC_PORT port, uint32_t pinMask, GPIO_DIR dir);

@brief Sets specific pins to a high logic level.
@param port The GPIO port (PORT_x [0...4]).
@param pinMask A 32-bit mask of the pins to set.
void GPIO_SetPins(LPC_PORT port, uint32_t pinMask);

@brief Clears specific pins to a low logic level.
@param port The GPIO port (PORT_x [0...4]).
@param pinMask A 32-bit mask of the pins to clear.
void GPIO_ClearPins(LPC_PORT port, uint32_t pinMask);

@brief Sets the logic state of a single pin.
@param port The GPIO port (PORT_x [0...4]).
@param pin The pin number (PIN_x [0...31]).
@param newState The desired state (SET for High, RESET for Low).
void GPIO_SetPinState(LPC_PORT port, LPC_PIN pin, SetState newState);

@brief Writes a 32-bit value to the entire port.
@param port The GPIO port (PORT_x [0...4]).
@param newValue The 32-bit value to write to the port.
void GPIO_WriteValue(LPC_PORT port, uint32_t newValue);

@brief Reads the current logic levels of all pins on a port.
@param port The GPIO port to read (PORT_x [0...4]).
@return A 32-bit value representing the state of all pins on the port.
uint32_t GPIO_ReadValue(LPC_PORT port);

@brief Toggles the logic state of specific pins.
@param port The GPIO port (PORT_x [0...4]).
@param pinMask A 32-bit mask of the pins to toggle.
void GPIO_TogglePins(LPC_PORT port, uint32_t pinMask);

@brief Configures the mask register for a port.
@param port The GPIO port (PORT_x [0...4]).
@param pinMask A 32-bit mask.
@param newState ENABLE to mask pins, DISABLE to unmask them.
void GPIO_SetMask(LPC_PORT port, uint32_t pinMask, FunctionalState newState);

@brief Configures the interrupt mask for an entire GPIO port.
@param port The GPIO port (Only PORT_0 and PORT_2 are supported).
@param newValue A 32-bit mask where each '1' enables the interrupt for that pin.
@param edgeState The edge to detect (GPIO_INT_RISING or GPIO_INT_FALLING).
void GPIO_IntConfigPort(LPC_PORT port, uint32_t newValue, GPIO_INT_EDGE edgeState);
```

@brief Enables or disables the interrupt for a specific GPIO pin and edge.
@param port The GPIO port (Only PORT_0 and PORT_2 are supported).
@param pin The pin number (PIN_x [0...31]).
@param edgeState The edge to detect (GPIO_INT_RISING or GPIO_INT_FALLING).
@param newState ENABLE to activate the interrupt, DISABLE to deactivate it.
*void **GPIO_IntConfigPin**(LPC_PORT port, LPC_PIN pin, GPIO_INT_EDGE edgeState, FunctionalState newState);*

@brief Retrieves the global interrupt status for a specific GPIO port.
@param port The GPIO port to check (Only PORT_0 and PORT_2 are supported).
@return SET if the port has a pending interrupt, RESET otherwise.
*SetState **GPIO_GetPortIntStatus**(LPC_PORT port);*

@brief Checks the interrupt status of a specific pin and edge.
@param port The GPIO port (Only PORT_0 and PORT_2 are supported).
@param pin The pin number (PIN_x [0...31]).
@param edgeState The edge status to verify (GPIO_INT_RISING or GPIO_INT_FALLING).
@return SET if the specific pin/edge interrupt is pending, RESET otherwise.
*SetState **GPIO_GetPinIntStatus**(LPC_PORT port, uint32_t pin, GPIO_INT_EDGE edgeState);*

@brief Clears the pending interrupt status for specific pins.
@param port The GPIO port (Only PORT_0 and PORT_2 are supported).
@param pinMask A 32-bit mask of the pins to clear.
*void **GPIO_ClearInt**(LPC_PORT port, uint32_t pinMask);*

EXTI

```
typedef enum {  
    EXTI_EINT0,  
    EXTI_EINT1,  
    EXTI_EINT2,  
    EXTI_EINT3  
} EXTI_LINE;
```

```
typedef enum {  
    EXTI_LEVEL_SENSITIVE,  
    EXTI_EDGE_SENSITIVE  
} EXTI_MODE;
```

```
typedef struct {  
    EXTI_LINE      line;  
    EXTI_MODE      mode;  
    EXTI_POLARITY  polarity;  
} EXTI_CFG_T;
```

```
typedef enum {  
    EXTI_LOW_ACTIVE    = 0,  
    EXTI_FALLING_EDGE = 0,  
    EXTI_HIGH_ACTIVE   = 1,  
    EXTI_RISING_EDGE   = 1  
} EXTI_POLARITY;
```

```
typedef enum {  
    EXTI_PULLUP = 0,  
    EXTI_PULLDOWN,  
    EXTI_NOPULL  
} EXTI_RESISTOR;
```

@brief Initializes the External Interrupt peripheral to a default state.
`void EXTI_Init(void);`

@brief Configures the physical pin and resistor mode for an EINT line.
@param line The external interrupt line (EXTI_EINTx [0...3]).
@param resMode The resistor configuration (EXTI_PULLUP, EXTI_PULLDOWN, or EXTI_NOPULL).
`void EXTI_PinConfig(EXTI_LINE line, EXTI_RESISTOR resMode);`

@brief Configures the trigger mode and polarity for an external interrupt.
@param extiCfg Pointer to an EXTI_CFG_T structure containing the settings.
`void EXTI_Config(const EXTI_CFG_T* extiCfg);`

@brief Configures and enables an external interrupt line in a single operation.
@param extiCfg Pointer to an EXTI_CFG_T structure containing the settings.
`void EXTI_ConfigEnable(const EXTI_CFG_T* extiCfg);`

@brief Clears the pending flag for a specific external interrupt line.
@param line The external interrupt line (EXTI_EINTx [0...3]).
`void EXTI_ClearFlag(EXTI_LINE line);`

@brief Retrieves the current status of an external interrupt flag.
@param line The external interrupt line to check (EXTI_EINTx [0...3]).
@return SET if the interrupt is pending, RESET otherwise.
`FlagStatus EXTI_GetFlag(EXTI_LINE line);`

@brief Clears pending requests and enables the interrupt in the NVIC.
@param line The external interrupt line to enable (EXTI_EINTx [0...3]).
`void EXTI_EnableIRQ(EXTI_LINE line);`

SYSTICK

@brief Initializes the SysTick timer using the internal processor clock.

@param time Desired interval in milliseconds.

```
void SYSTICK_InternalInit(uint32_t time);
```

@brief Initializes the SysTick timer using an external clock source.

@param extFreq Frequency of the external clock source in Hz.

@param time Desired interval in milliseconds.

```
void SYSTICK_ExternalInit(uint32_t extFreq, uint32_t time);
```

@brief Enables or disables the SysTick counter.

@param newState ENABLE to start the counter, DISABLE to stop it.

```
void SYSTICK_Cmd(FunctionalState newState);
```

@brief Enables or disables the SysTick exception request.

@param newState ENABLE to activate interrupts, DISABLE to deactivate them.

```
void SYSTICK_IntCmd(FunctionalState newState);
```

@brief Retrieves the current value of the SysTick counter.

@return Current value of the 24-bit counter.

```
uint32_t SYSTICK_GetCurrentValue(void);
```

@brief Clears the SysTick Count Flag.

```
void SYSTICK_ClearCounterFlag(void);
```

@brief Retrieves the current reload (LOAD) value.

@return The 24-bit reload value.

```
uint32_t SYSTICK_GetReloadValue(void);
```

@brief Sets a new reload value for the counter.

@param reloadTicks The new 24-bit reload value (Max 0xFFFFFF).

```
void SYSTICK_SetReloadValue(uint32_t reloadTicks);
```

@brief Checks if the SysTick counter is currently enabled.

@return SET if the counter is enabled, RESET otherwise.

```
FlagStatus SYSTICK_IsActive(void);
```

@brief Checks if the SysTick timer has counted to zero since the last read.

@return SET if the timer has fired, RESET otherwise.

```
FlagStatus SYSTICK_HasFired(void);
```

TIMERS

```
typedef enum {
    TIM_TIMER_MODE = 0,
    TIM_COUNTER_MODE
} TIM_MODE;

typedef enum {
    TIM_CTR_RISING = 1,
    TIM_CTR_FALLING,
    TIM_CTR_ANY
} TIM_CTR_EDGE;

typedef enum {
    TIM_CAPTURE_0 = 0,
    TIM_CAPTURE_1
} TIM_CAPTURE_CH;

typedef enum {
    TIM_CAP_NONE = 0,
    TIM_CAP_RISING,
    TIM_CAP_FALLING,
    TIM_CAP_ANY
} TIM_CAP_MODE;

typedef enum {
    TIM_MATCH_0 = 0,
    :
    TIM_MATCH_3
} TIM_MATCH_CH;

typedef enum {
    TIM_NOTHING = 0,
    TIM_LOW,
    TIM_HIGH,
    TIM_TOGGLE
} TIM_EXTMATCH_OPT;

typedef struct {
    TIM_PRESCALE           prescaleOpt;
    uint32_t               prescaleValue;
} TIM_TIMERCFG_T;

typedef struct {
    TIM_CAPTURE_CH         input;
    TIM_CTR_EDGE           edge;
} TIM_COUNTERCFG_T;

typedef struct {
    TIM_CAPTURE_CH         channel;
    FunctionalState         risingEn;
    FunctionalState         fallingEn;
    FunctionalState         intEn;
} TIM_CAPTURECFG_T;
```

```
typedef enum {
    TIM_TICK = 0,
    TIM_US
} TIM_PRESCALE;

typedef enum {
    TIM_MR0_INT = 0,
    :
    TIM_MR3_INT,
    TIM_CR0_INT,
    TIM_CR1_INT
} TIM_INT;

typedef enum {
    TIM_CAP0_0_P1_26 = 0,
    TIM_CAP0_1_P1_27,
    TIM_MAT0_0_P1_28,
    TIM_MAT0_0_P3_25,
    TIM_MAT0_1_P1_29,
    TIM_MAT0_1_P3_26,
    TIM_CAP1_0_P1_18,
    TIM_CAP1_1_P1_19,
    TIM_MAT1_0_P1_22,
    TIM_MAT1_1_P1_25,
    TIM_CAP2_0_P0_4,
    TIM_CAP2_1_P0_5,
    TIM_MAT2_0_P0_6,
    TIM_MAT2_0_P4_28,
    TIM_MAT2_1_P0_7,
    TIM_MAT2_1_P4_29,
    TIM_MAT2_2_P0_8,
    TIM_MAT2_3_P0_9,
    TIM_CAP3_0_P0_23,
    TIM_CAP3_1_P0_24,
    TIM_MAT3_0_P0_10,
    TIM_MAT3_1_P0_11
} TIM_PIN_OPTION;
```

```

typedef struct {
    TIM_MATCH_CH      channel;
    FunctionalState    intEn;
    FunctionalState    stopEn;
    FunctionalState    resetEn;
    TIM_EXTMATCH_OPT   extOpt;
    uint32_t           matchValue;
} TIM_MATCHCFG_T;

    @brief Initializes the timer in Timer Mode.
    @param TIMx Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @param timerCfg Pointer to a TIM_TIMERCFG_T structure with prescale settings.
void TIM_InitTimer(LPC_TIM_TypeDef* TIMx, const TIM_TIMERCFG_T* timerCfg);

    @brief Initializes the timer in Counter Mode.
    @param TIMx Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @param counterCfg Pointer to a TIM_COUNTERCFG_T structure specifying the input
        channel and edge
void TIM_InitCounter(LPC_TIM_TypeDef* TIMx, const TIM_COUNTERCFG_T* counterCfg);

    @brief De-initializes the timer peripheral.
    @param TIMx Pointer to the timer peripheral to disable (LPC_TIMx [0...3]).
void TIM_DeInit(LPC_TIM_TypeDef* TIMx);

    @brief Starts the timer counter.
    @param TIMx Pointer to the timer peripheral (LPC_TIMx [0...3]).
void TIM_Enable(LPC_TIM_TypeDef* TIMx);

    @brief Stops the timer counter.
    @param TIMx Pointer to the timer peripheral (LPC_TIMx [0...3]).
void TIM_Disable(LPC_TIM_TypeDef* TIMx);

    @brief Retrieves the current value of the Timer Counter (TC).
    @param TIMx Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @return Current 32-bit counter value.
uint32_t TIM_ReadTimer(LPC_TIM_TypeDef* TIMx);

    @brief Retrieves the current value of the Prescale Counter (PC).
    @param TIMx Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @return Current 32-bit prescale counter value.
uint32_t TIM_ReadPrescale(LPC_TIM_TypeDef* TIMx);

    @brief Resets the Timer Counter and Prescale Counter.
    @param TIMx Pointer to the timer peripheral (LPC_TIMx [0...3]).
void TIM_ResetCounter(LPC_TIM_TypeDef* TIMx);

    @brief Configures the match logic and external output for a timer channel.
    @param TIMx Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @param matchCfg Pointer to a TIM_MATCHCFG_T structure with match settings.
void TIM_ConfigMatch(LPC_TIM_TypeDef* TIMx, const TIM_MATCHCFG_T* matchCfg);

```

```

    @brief Updates the value of a specific Match Register.
    @param TIMx      Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @param channel    The match channel (TIM_MATCH_x [0...3]).
    @param matchValue The new 32-bit match value.
void TIM_UpdateMatchValue(LPC_TIM_TypeDef* TIMx, TIM_MATCH_CH channel, uint32_t
matchValue);

    @brief Configures the behavior of the external match pins (MATn.x).
    @param TIMx      Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @param channel    The match channel associated with the MAT pin (TIM_MATCH_x
        [0...3]).
    @param type      The external match operation (TIM_NOTHING, TIM_LOW, TIM_HIGH,
        TIM_TOGGLE).
void TIM_SetMatchExt(LPC_TIM_TypeDef* TIMx, TIM_MATCH_CH channel, TIM_EXTMATCH_OPT
type);

    @brief Configures the capture logic for external signal timing.
    @param TIMx      Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @param capCfg    Pointer to a TIM_CAPTURECFG_T structure with capture settings.
void TIM_ConfigCapture(LPC_TIM_TypeDef* TIMx, const TIM_CAPTURECFG_T* capCfg);

    @brief Retrieves the last value stored in a Capture Register.
    @param TIMx      Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @param channel    The capture channel to read (TIM_CAPTURE_0 or TIM_CAPTURE_1).
    @return The 32-bit captured Timer Counter value.
uint32_t TIM_GetCaptureValue(LPC_TIM_TypeDef* TIMx, TIM_CAPTURE_CH channel);

    @brief Clears a pending interrupt flag for a timer event.
    @param TIMx      Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @param intFlag    The interrupt source to clear (TIM_xxx_INT).
void TIM_ClearIntPending(LPC_TIM_TypeDef* TIMx, TIM_INT intFlag);

    @brief Retrieves the status of a specific timer interrupt flag.
    @param TIMx      Pointer to the timer peripheral (LPC_TIMx [0...3]).
    @param intFlag    The interrupt source to check (TIM_xxx_INT).
    @return SET if the interrupt is pending, RESET otherwise.
FlagStatus TIM_GetIntStatus(LPC_TIM_TypeDef* TIMx, TIM_INT intFlag);

    @brief Configures the physical pin for a specific Timer function.
    @param option    Selection from the available TIM_PIN_OPTION values.
void TIM_PinConfig(TIM_PIN_OPTION option);

```

ADC

```
typedef enum {
    ADC_CHANNEL_0 = 0,
    :
    ADC_CHANNEL_7
} ADC_CHANNEL;

typedef enum {
    ADC_INT_CH0 = 0,
    :
    ADC_INT_CH7,
    ADC_INT_GLOBAL
} ADC_INT_SOURCE;

typedef enum {
    ADC_START_ON_RISING = 0,
    ADC_START_ON_FALLING
} ADC_START_ON_EDGE;

    @brief Initializes the ADC peripheral with a specific sampling rate.
    @param rate Desired sampling rate in Hz (e.g., 100000 for 100kHz).
void ADC_Init(uint32_t rate);

    @brief De-initializes the ADC peripheral.
void ADC_DeInit(void);

    @brief Configures the physical pin associated with an ADC channel.
    @param channel ADC channel to be configured (ADC_CHANNEL_x [0...7]).
void ADC_PinConfig(ADC_CHANNEL channel);

    @brief Enables the ADC Burst conversion mode.
void ADC_BurstEnable(void);

    @brief Disables the ADC Burst conversion mode.
void ADC_BurstDisable(void);

    @brief Powers up the ADC converter.
void ADC_PowerUp(void);

    @brief Powers down the ADC converter.
void ADC_PowerDown(void);

    @brief Configures the start mode or trigger for the ADC conversion.
    @param mode Selected trigger source:
void ADC_StartCmd(ADC_START_MODE mode);

    @brief Selects an ADC channel for the next conversion sequence.
    @param channel The ADC channel to enable (ADC_CHANNEL_x [0...7]).
void ADC_ChannelEnable(ADC_CHANNEL channel);

    @brief De-selects an ADC channel from the conversion sequence.
    @param channel The ADC channel to disable (ADC_CHANNEL_x [0...7]).
void ADC_ChannelDisable(ADC_CHANNEL channel);

typedef enum {
    ADC_START_CONTINUOUS = 0,
    ADC_START_NOW,
    ADC_START_ON_EINT0,
    ADC_START_ON_CAP01,
    ADC_START_ON_MAT01,
    ADC_START_ON_MAT03
} ADC_START_MODE;

typedef enum {
    ADC_DATA_OVERRUN = 0,
    ADC_DATA_DONE
} ADC_DATA_STATUS;
```

```

    @brief Configures the trigger edge for external hardware starts.
    @param edge The desired edge (ADC_START_ON_RISING or ADC_START_ON_FALLING).
void ADC_EdgeStartConfig(ADC_START_ON_EDGE edge);

    @brief Enables an interrupt source for the ADC.
    @param source The interrupt source to enable (ADC_INT_CHx [0...7] or
ADC_INT_GLOBAL).
void ADC_IntEnable(ADC_INT_SOURCE source);

    @brief Disables an interrupt source for the ADC.
    @param source The interrupt source to disable (ADC_INT_CHx [0...7] or
ADC_INT_GLOBAL).
void ADC_IntDisable(ADC_INT_SOURCE source);

    @brief Reads the global status of the most recent ADC conversion.
    @param type Status type to check (ADC_DATA_DONE or ADC_DATA_OVERRUN).
    @return SET if the flag is active, RESET otherwise.
FlagStatus ADC_GlobalGetStatus(ADC_DATA_STATUS type);

    @brief Reads the status of a specific ADC channel.
    @param channel The ADC channel to check (ADC_CHANNEL_x [0...7]).
    @param type Status type (DONE or OVERRUN).
    @return SET if the flag is active, RESET otherwise.
FlagStatus ADC_ChannelGetStatus(ADC_CHANNEL channel, ADC_DATA_STATUS type);

    @brief Retrieves the result of the last conversion from any channel.
    @return The 12-bit conversion result (0 to 4095).
uint16_t ADC_GlobalGetData(void);

    @brief Retrieves the conversion result for a specific ADC channel.
    @param channel The ADC channel to read (ADC_CHANNEL_x [0...7]).
    @return The 12-bit conversion result (0 to 4095).
uint16_t ADC_ChannelGetData(ADC_CHANNEL channel);

```

DAC

```
typedef enum {  
    DAC_700uA = 0,  
    DAC_350uA  
} DAC_MAX_CURRENT;
```

```
typedef struct {  
    FunctionalState doubleBuffer;  
    FunctionalState dmaCounter;  
    FunctionalState dmaRequest;  
} DAC_CONVERTER_CFG_T;
```

```
    @brief Initializes the DAC peripheral and its associated pin.  
void DAC_Init(void);
```

```
    @brief Updates the analog output value.  
    @param newValue 10-bit digital value (0 to 1023).  
void DAC_UpdateValue(uint32_t newValue);
```

```
    @brief Sets the maximum update rate and settling time.  
    @param maxCurr Power mode selection (DAC_700uA or DAC_350uA).  
void DAC_SetBias(DAC_MAX_CURRENT maxCurr);
```

```
    @brief Configures the DAC control register for advanced features.  
    @param dacCfg Pointer to a DAC_CONVERTER_CFG_T structure containing the  
        settings.  
void DAC_ConfigDACConverterControl(const DAC_CONVERTER_CFG_T* dacCfg);
```

```
    @brief Sets the reload value for the DAC timeout counter.  
    @param timeOut 16-bit timeout value in PCLK cycles.  
void DAC_SetDMATimeOut(uint16_t timeOut);
```

GPDMA

```
typedef enum {  
    GPDMA_CH_0 = 0,  
    :  
    GPDMA_CH_7  
} GPDMA_CH;
```

```
typedef enum {  
    GPDMA_SSP0_Tx = 0,  
    GPDMA_SSP0_Rx,  
    GPDMA_SSP1_Tx,  
    GPDMA_SSP1_Rx,  
    GPDMA_ADC,  
    GPDMA_I2S_C0,  
    GPDMA_I2S_C1,  
    GPDMA_DAC,  
    GPDMA_UART0_Tx,  
    GPDMA_UART0_Rx,  
    GPDMA_UART1_Tx,  
    GPDMA_UART1_Rx,  
    GPDMA_UART2_Tx,  
    GPDMA_UART2_Rx,  
    GPDMA_UART3_Tx,  
    GPDMA_UART3_Rx,  
    GPDMA_MAT0_0,  
    GPDMA_MAT0_1,  
    GPDMA_MAT1_0,  
    GPDMA_MAT1_1,  
    GPDMA_MAT2_0,  
    GPDMA_MAT2_1,  
    GPDMA_MAT3_0,  
    GPDMA_MAT3_1  
} GPDMA_CONNECTION;
```

```
typedef enum {  
    GPDMA_CLR_INTTC,  
    GPDMA_CLR_INTERR  
} GPDMA_CLEAR_INT;
```

```
typedef struct {  
    GPDMA_TRANSFER_WIDTH    width;  
    GPDMA_BURST_SIZE        burst;  
    FunctionalState          increment;  
} GPDMA_Endpoint_T;
```

```
typedef struct {  
    uint32_t    srcAddr;  
    uint32_t    dstAddr;  
    uint32_t    nextLLI;  
    uint32_t    control;  
} GPDMA_LLI_T;
```

```
typedef enum {  
    GPDMA_M2M = 0,  
    GPDMA_M2P,  
    GPDMA_P2M,  
    GPDMA_P2P  
} GPDMA_TRANSFER_TYPE;
```

```
typedef enum {  
    GPDMA_BSIZE_1 = 0,  
    GPDMA_BSIZE_4,  
    GPDMA_BSIZE_8,  
    GPDMA_BSIZE_16,  
    GPDMA_BSIZE_32,  
    GPDMA_BSIZE_64,  
    GPDMA_BSIZE_128,  
    GPDMA_BSIZE_256,  
    GPDMA_BURST_AUTO  
} GPDMA_BURST_SIZE;
```

```
typedef enum {  
    GPDMA_BYTE = 0,  
    GPDMA_HALFWORD,  
    GPDMA_WORD,  
    GPDMA_WIDTH_AUTO  
} GPDMA_TRANSFER_WIDTH;
```

```
typedef enum {  
    GPDMA_INT,  
    GPDMA_INTTC,  
    GPDMA_INTERR,  
    GPDMA_RAW_INTTC,  
    GPDMA_RAW_INTERR,  
    GPDMA_ENABLED_CH  
} GPDMA_STATUS_TYPE;
```

```

typedef struct {
    GPDMA_CH                channelNum;
    uint32_t                transferSize;
    GPDMA_TRANSFER_TYPE     type;
    uint32_t                srcMemAddr;
    uint32_t                dstMemAddr;
    GPDMA_CONNECTION        srcConn;
    GPDMA_CONNECTION        dstConn;
    GPDMA_Endpoint_T        src;
    GPDMA_Endpoint_T        dst;
    FunctionalState          intTC;
    FunctionalState          intErr;
    uint32_t                linkedList;
} GPDMA_Channel_CFG_T;

    @brief Initializes the GPDMA controller and resets all channels.
void GPDMA_Init(void);

    @brief De-initializes the GPDMA peripheral.
void GPDMA_DeInit(void);

    @brief Configures a specific GPDMA channel for a data transfer.
    @param dmaCfg Pointer to a GPDMA_Channel_CFG_T structure containing all transfer
        parameters.
    @return SUCCESS if the configuration was applied, ERROR if the channel is busy or
Status GPDMA_SetupChannel(const GPDMA_Channel_CFG_T* dmaCfg);

    @brief Starts the DMA transfer on the specified channel.
    @param channel The GPDMA channel to start (GPDMA_CHANNEL_x [0...7]).
void GPDMA_ChannelStart(GPDMA_CH channel);

    @brief Forces an immediate stop of the DMA transfer.
    @param channel The GPDMA channel to stop (GPDMA_CHANNEL_x [0...7]).
void GPDMA_ChannelStop(GPDMA_CH channel);

    @brief Stops the DMA transfer after finishing the current transaction.
    @param channel The GPDMA channel to stop gracefully (GPDMA_CHANNEL_x [0...7]).
void GPDMA_ChannelGracefulStop(GPDMA_CH channel);

    @brief Suspends the DMA transfer on the specified channel.
    @param channel The GPDMA channel to pause (GPDMA_CHANNEL_x [0...7]).
void GPDMA_ChannelPause(GPDMA_CH channel);

    @brief Resumes a previously paused DMA transfer.
    @param channel The GPDMA channel to resume (GPDMA_CHANNEL_x [0...7]).
void GPDMA_ChannelResume(GPDMA_CH channel);

    @brief Retrieves the current status of a specific GPDMA interrupt or channel
        state.
    @param type The type of status to retrieve:
    @param channel The GPDMA channel to check (GPDMA_CH_x [0...7]).
    @return SET if the specific status bit is active, RESET otherwise.
IntStatus GPDMA_IntGetStatus(GPDMA_STATUS_TYPE type, GPDMA_CH channel);

    @brief Clears the pending interrupt flags for a specific GPDMA channel.
    @param type The interrupt flag to clear (GPDMA_CLR_INTTC or GPDMA_CLR_INTERR).
    @param channel The GPDMA channel affected (GPDMA_CH_x [0...7]).
void GPDMA_ClearIntPending(GPDMA_CLEAR_INT type, GPDMA_CH channel);

```